

The Codebase, File by File

What every file in the project does, and why it exists

Project — Trisheet: company profiles, sourced from filings

Covers — 31 backend files, 73 frontend files, 8 database tables

Stack — Next.js 15 + TypeScript (frontend), FastAPI + Python 3.11 (backend), Supabase (database)

Date — 5 August 2026

How to read this document

This document is a map. It lists every file in the project and explains in plain English what that file is responsible for. You should be able to open any file in the codebase, find it here, and understand what it is for without reading its code.

The project is split into two halves that run as separate programs:

- **The frontend** — the website you see in the browser. Written in TypeScript using Next.js. It draws the screens and never does any financial work of its own.
- **The backend** — the engine. Written in Python using FastAPI. It fetches filings from the government, does all the maths, asks the language model for prose, checks the result, and stores it.

They talk to each other over HTTP. The browser never talks to the SEC or to the language model directly — everything goes through the backend, which is what makes the rules in this project enforceable.

One naming convention to know. The backend's main logic lives in thirteen files named `m01` through `m13`. The numbers are the order they run in. This is deliberate: you can read the pipeline top to bottom and each module only ever uses what the earlier ones produced. No module reaches backwards.

Repository map

trishet/		
frontend/	the website	→ deployed to Vercel
backend/	the engine	→ deployed to Railway
db/schema.sql	the database structure	→ runs on Supabase
docs/	design and decision notes	
CLAUDE.md	the project's binding rules	
README.md	setup instructions	
CHANGELOG.md	history of changes	

Part 1 — The backend

Thirty-one Python files, about 24,000 lines. Grouped into three layers: the application core (wiring), the modules (the actual work), and the services (talking to the outside world).

1.1 Application core

These files hold the app together. None of them knows anything about filings or financial figures — that knowledge lives in the modules.

FILE	LINES	WHAT IT DOES
<code>app/main.py</code>	550	The front door. Defines every web address the backend answers on (<code>/health</code> , <code>/resolve</code> , <code>/reports</code> , and so on). It is wiring only — it contains no report logic at all. Its most important design decision: generating a report takes minutes, far longer than a browser will wait, so asking for a report does not wait for one. It registers the job, immediately hands back a job ID, and the work continues in the background.
<code>app/pipeline.py</code>	774	The conductor. Runs the thirteen modules in order, moves the report through its statuses, and records what each step produced and how long it took. It holds no opinion about the content. It also defines the failure policy in one place: the SEC is the only thing that cannot fail; everything else is allowed to produce nothing and the report still finishes.
<code>app/models.py</code>	1,290	The shapes of all data. Defines every object that crosses a boundary. The most important one is <code>Fact</code> — a single sourced figure. Its five provenance fields have no default values, which means a fact that cannot name its source physically cannot be created. That is a property of the type, not a check someone must remember to run.
<code>app/config.py</code>	2,585	Every setting in one place. 217 named constants. No number or text literal is allowed anywhere else in the codebase. This is where the XBRL tag fallback lists live, the tier rules, the rate limits, the prompt templates and the depth profiles. Secrets are read from the environment and wrapped so an accidental log line cannot leak them.
<code>app/logging_config.py</code>	121	Structured logging. Every log line is a single line of JSON so hosting platforms can parse it. The report ID is carried automatically, so every line written while handling a report is tagged with which report it belongs to. <code>print()</code> is never used anywhere in the codebase.
<code>app/cli.py</code>	525	Command line tool. Lets an operator run the pipeline without the website — useful for testing and debugging. This is the single place in the codebase where writing to the screen is the point rather than a mistake.
<code>app/__init__.py</code>	1	Marks the folder as a Python package.

1.2 The thirteen modules — where the work happens

Each module has one job and one documented interface. Below, each is described in the order it runs.

mo1_resolver.py — who is this company? (450 lines)

Turns what the user typed — a ticker like `AAPL` or a name like "Taiwan Semiconductor" — into a confirmed company identity: its CIK (the SEC's ID number, always padded to 10 digits), whether it is a US or foreign filer, its industry code, its financial year end and the currency it reports in.

Two decisions matter here. Filer type is decided by looking at the annual form the company actually filed most recently, not by guessing from its address. And if the input matches more than one company, it returns the list of candidates and picks nothing — guessing which company the reader meant would produce a correct-looking report about the wrong business.

mo2_discovery.py — which filings may we use? (412 lines)

Takes the company's entire filing history and narrows it to the documents this report is allowed to draw on. Three things happen so no later module has to repeat them: irrelevant forms (insider trades, employee share plans) are dropped; amendments take precedence over the originals they replace, because a restated figure is the correct one; and every filing gets a full absolute URL and accession number attached.

mo3_financials.py — the numbers (1,118 lines)

Pulls reported figures out of the SEC's XBRL data — revenue, operating income, assets, cash flow and so on — and turns each into a `Fact` with full provenance and a record of which tag actually produced it.

Its rules: never rely on a single XBRL tag, because companies tag the same concept differently; each metric has an ordered list of fallbacks. Try the US accounting taxonomy first, fall back to the international one for foreign filers. Remove duplicates, keeping the most recently filed. Prefer amendments. And a metric that cannot be found anywhere produces a "not disclosed" marker — never an estimate, never an interpolation.

mo4_narrative.py — the words (607 lines)

Extracts the text sections of the annual report: the business description, the risk factors, and management's discussion. It does not summarise, rank or interpret. A risk is stated as the company headed it. This text is later shown to the language model rather than the model being asked to recall anything, which is what keeps the written sections sourced.

mo5_market.py — share price and valuation (493 lines)

This is the only file in the entire project permitted to fetch market data. If any other file reached a market data provider, that would be a bug. This single choke point is the physical expression of the source-tier rule — everything this module produces is Tier 3 and is therefore banned from the financial highlights section by two independent later checks.

Fetches current share price, market capitalisation and trading multiples. If the market data provider is down, the report still generates without the valuation table.

mo6_factstore.py — the write gate (529 lines)

The wall. Every single fact in the system passes through this file, and this is where provenance is enforced in code rather than by prompting or convention.

Two refusals happen here, and both are logged rather than silently swallowed:

- A fact missing any provenance field — tier, source type, source URL, accession number or filing date — is discarded. It is never stored blank.
- A Tier 3 or Tier 4 fact offered for a financial-highlights metric is hard-blocked. Market data and news cannot enter the financial statements section, by construction.

It deliberately accepts raw untyped data rather than only well-formed objects. The reason is subtle and important: a `Fact` object cannot be built without provenance, so a gate that only accepted `Fact` objects would never reject anything and the wall would be somewhere else. This gate checks what actually arrived.

m07_analysis.py — all the arithmetic (3,231 lines)

Every calculation in the entire system happens in this one file. The language model never performs arithmetic. It receives what this module computed and writes prose about it.

Computes growth rates, margins, ratios, bridges, discounted cash flow valuations, sensitivity tables and scenario analysis using pandas and numpy. Every function is pure — no file access, no global state, no side effects — which means identical input always produces byte-identical output. That is what makes a report reproducible. Every derived value carries the formula that produced it, so the interface can show its working.

m08_peers.py — who are the comparable companies? (1,562 lines)

Picks comparable companies using a ladder of decreasing authority, and records which rung answered:

1. **The compensation peer group in the proxy statement.** A pay committee that benchmarks executive salaries against a named group has stated on the record who it believes its peers are. Nothing the system could infer beats that.
2. **The competition section of the annual report** — the company naming its own competitors, in its own words.
3. **Industry code match** — the SEC's own classification.

m09_developments.py — recent events (462 lines)

Builds a dated timeline from current-report filings (8-K for US companies, 6-K for foreign ones), each entry sourced to the filing it came from. It filters by the filing's own item numbers — a material agreement, a completed acquisition, results of operations, a change of officers — and drops everything else.

m10_writer.py — the language model writes (393 lines)

Hands the model a set of already-computed facts and asks it to write the narrative around them. What reaches the prompt is `Fact` objects and nothing else — every one has already passed the provenance gate. No raw web text, no filing HTML and no market payload is ever put into a prompt in this module.

m11_factcheck.py — the verification gate (900 lines)

The audit. This module is the reason the product's claim is true rather than merely intended. Everything upstream is a control; this is the check on them. Violations are graded: blocking findings mark the report as failed verification, and each is shown beside the figure it concerns rather than hidden.

It checks that every number appearing in the prose exists in the fact store — a figure matching nothing is a fabrication, whatever produced it. It checks that every figure is supported by a citation. It checks segment

figures sum to the consolidated total within tolerance, and that assets equal liabilities plus equity.

m12_assembler.py — the outputs (3,092 lines)

Turns a verified report into the three forms it is consumed in: the JSON document the browser renders, a PDF styled as an analyst tearsheet, and an Excel workbook whose ratios are live formulas over the reported figures rather than pasted values. The canonical document is built once and all three are rendered from it, so a figure cannot appear in the PDF having failed verification for the screen.

m13_sources.py — links the reader attached (295 lines)

If the reader attaches a link — an investor relations page, a press release — this module reads it and quotes what it says. Crucially, it cannot produce a `Fact`. It produces a separate `SourceNote` type instead, because a company web page would otherwise pass the tier rules and be rendered in the financial highlights table beside figures traceable to filings. The distinction is enforced by the type, not by a reviewer's care.

1.3 Services — talking to the outside world

FILE	LINES	WHAT IT DOES
<code>services/edgar.py</code>	425	The SEC client. Every request to the government's servers goes through here; nothing else in the codebase builds an SEC URL. It enforces the SEC's rules: send an identifying User-Agent (they return an error without one), never exceed 10 requests per second across all workers using a shared token bucket, retry rate-limit responses with backoff up to three attempts, and cache filing responses permanently because filings never change.
<code>services/llm.py</code>	548	The Anthropic client. The only file that talks to the language model. It transports prompts and returns text and holds no opinion about content. Output is constrained to a JSON schema the caller supplies, so the model answers in a shape that can be verified rather than in free prose. Randomness is held low so results are near-reproducible.
<code>services/db.py</code>	109	The database client. The only file that opens a database connection. It uses the service-role key which bypasses row-level security — which is exactly why nothing reachable from the browser may import it.
<code>services/runlog.py</code>	725	The run record. Everything a running report writes about <i>itself</i> — its status, its steps, timings and artifacts. Two consumers read it: the browser's live progress feed, and the metrics endpoint.
<code>services/storage.py</code>	165	File uploads. The only file that uploads a rendered PDF or spreadsheet. Storage is treated as a sink, not a source: a report whose PDF failed to upload is a complete report with a missing download link, and the interface says so. It is never a failed report.
<code>services/document.py</code>	234	Filing text extraction. Turns the HTML the SEC serves into readable text and sentences. Two modules need this, so neither carries its own parser. It is explicitly not a fact source — provenance is attached by whichever module called it.
<code>services/webfetch.py</code>	205	Fetching a user-supplied page. The only module that opens a connection to an address a user typed rather than to a fixed known host. Every other network client points at exactly one place and never needs to defend against where it is aimed. This one is aimed somewhere new on every call, so it carries its own guards.
<code>services/metrics.py</code>	187	Operational statistics. Success rate, latency, citation coverage and cost, counted from actual run records rather than sampled. Notably: a period with no runs reports "no data" rather than "0%", because those are different statements.
<code>modules/chat_agent.py</code>	2,056	The report assistant. Answers a reader's question about a finished report, using only facts that already carry provenance. It works in tiers, stopping at the first that has an answer: first the facts already stored for this report (matched by plain Python word overlap, not by asking the model to decide), then metrics derivable from them. Questions asking for investment advice are redirected rather than answered.

Part 2 — The frontend

Seventy-three TypeScript files. The website has two visual identities on purpose: the landing page is colourful and modern to invite a first-time visitor in; the report itself is austere and paper-like, because it must read as a research document rather than a dashboard.

2.1 Routes — the actual pages

FILE	LINES	WHAT IT DOES
<code>app/layout.tsx</code>	52	The shell every page sits inside — fonts, colours, page metadata.
<code>app/page.tsx</code>	12	The home page at <code>/</code> . Renders the input screen.
<code>app/r/[id]/page.tsx</code>	49	The report page at <code>/r/<id></code> . Loads a generated report by its ID and renders it.
<code>app/preview/page.tsx</code>	103	An index of the preview routes, used for interface testing.
<code>app/preview/input/page.tsx</code>	141	The input screen with test data.
<code>app/preview/progress/[script]/page.tsx</code>	67	Replays a scripted pipeline run — normal, degraded and failed — so the progress screen can be verified without a backend.
<code>app/preview/report/[fixture]/page.tsx</code>	31	Renders a full report from fixture data. Two fixtures: a US filer and a foreign filer.

2.2 Shared logic (lib)

FILE	LINES	WHAT IT DOES
<code>lib/types.ts</code>	648	Every shared data shape, mirroring the backend's models. When one changes, both change.
<code>lib/api.ts</code>	226	The typed client for the backend. Transport only — no business logic, no formatting. Every call returns a success-or-failure result rather than throwing.
<code>lib/provenance.ts</code>	150	The join between figures and their sources. Builds the mapping that gives every figure a superscript marker pointing at a card in the provenance rail. Pure logic, nothing else.
<code>lib/format.ts</code>	157	Turns final values into display strings. It never scales, sums or divides — every figure arrives already computed by the backend.
<code>lib/constants.ts</code>	129	Every piece of text the interface shows, in one reviewable place. Nothing hardcoded to one company.
<code>lib/supabase.ts</code>	32	The browser's database connection — reads and live updates only. All writes go through the backend so provenance rules cannot be bypassed from the browser.

lib/logo.ts	24	Company logos by ticker. Returns nothing when unconfigured so the caller falls back gracefully — a logo is decoration, never a dependency.
lib/utils.ts	6	A small helper for combining CSS class names.
lib/mock/*.ts	2,425	Six files of fixture data used only by the preview routes. Every one carries a header stating the figures are placeholders and not sourced from filings, so a screenshot can never be mistaken for a real report.

2.3 Report components — the research document

FILE	LINES	WHAT IT DOES
report/report-view.tsx	123	Assembles the whole report page from its sections.
report/report-header.tsx	128	The masthead. States who the filer is, which annual form they file and what currency they report in — before any figure appears, because a reader who does not know whether this is a 10-K or a 20-F cannot read the rest correctly.
report/provenance-rail.tsx	223	The signature element. The persistent column of source cards beside the report. Each shows form type, accession number, filing date and a link. It is never collapsed into a footer.
report/provenance-context.tsx	103	Carries the figure-to-source mapping down the component tree so hovering a figure can highlight its source.
report/figure.tsx	155	A single figure with its citation marker. Right-aligned, monospace, tabular numerals.
report/figure-table.tsx	101	A table of figures across periods.
report/compliance-strip.tsx	288	What the verification gate found: how many facts the report rests on, how they spread across the four source tiers, and how many figures carry a citation. Every number arrives already computed; the only arithmetic here is turning a count into a bar width.
report/report-section.tsx	99	The wrapper around each numbered section.
report/prose.tsx	58	Renders written paragraphs with their inline citation markers.
report/risk-list.tsx	86	Risk factors as the filer headed them.
report/developments-timeline.tsx	114	The dated timeline of recent filings.
report/source-notes.tsx	79	Statements read from links the reader attached — rendered apart from and after every other section, headed "supplied" and marked "not verified", because they are not part of the filed record.
report/section-sidebar.tsx	89	The section navigation list.
report/artifact-downloads.tsx	100	The PDF and Excel download links.
report/company-logo.tsx	39	The company mark, with a graceful fallback.
report/report-unavailable.tsx	40	What renders when there is no report. States what happened and what to do, in the interface's voice — never a stack trace, never a blank screen.
report/chat-panel.tsx	290	The assistant panel where a reader asks questions about the report.
report/chat-message.tsx	169	One message in that conversation, with its citations.
report/chat-composer.tsx	110	The question input box.

2.4 Input screen components

FILE	LINES	WHAT IT DOES
<code>input/input-screen.tsx</code>	251	The landing page layout.
<code>input/ticker-form.tsx</code>	299	The search form that starts a report.
<code>input/ticker-combobox.tsx</code>	271	The search box with live ticker suggestions as you type.
<code>input/disambiguation.tsx</code>	106	Shown when a search matches several companies — the reader chooses, the system never guesses.
<code>input/depth-selector.tsx</code> / <code>depth-menu.tsx</code>	390	Choosing how deep the analysis goes. Depth never changes a provenance rule — only how many periods are examined.
<code>input/report-preview.tsx</code>	86	A sample of a finished report, labelled "Sample report" throughout, so a first-time visitor sees the citation pattern before generating anything.
<code>input/trust-section.tsx</code>	57	Three claims a reader can verify. Each names the control that makes it true rather than asserting the outcome alone.
<code>input/monolith-canvas.tsx</code>	846	The animated hero graphic on the landing page.
<code>input/hero-illustration.tsx</code>	67	A decorative graphic, marked as decorative for screen readers. Carries no real figures.
<code>input/example-chips.tsx</code>	47	Example tickers a visitor can click to start.
<code>input/ai-financial-agent-*.tsx</code>	164	The section showcasing the assistant on the landing page.
<code>input/monolith-annotations.tsx</code>	154	Labels over the hero graphic.

2.5 Charts, progress, and shared parts

FILE	LINES	WHAT IT DOES
<code>charts/revenue-margin-chart.tsx</code>	136	Revenue and margin over time.
<code>charts/cash-flow-chart.tsx</code>	136	Cash flow across periods.
<code>charts/segment-mix-chart.tsx</code>	114	Revenue split by business segment.
<code>charts/peer-valuation-chart.tsx</code>	129	Valuation against comparable companies.
<code>charts/chart-primitives.tsx</code>	206	Shared axes, gridlines and tooltips so all four charts match.
<code>progress/progress-screen.tsx</code>	76	The live screen shown while a report generates.
<code>progress/step-feed.tsx</code>	165	The list of pipeline steps updating in real time, each saying what it produced.
<code>chrome/site-header.tsx</code>	114	The header, in two variants so the wordmark is identical on both surfaces — dark over the landing hero, hairline-bordered over the report.

<code>preview/preview-notice.tsx</code>	33	The standing banner on every preview page, in the colour reserved for things needing a flag, so a screenshot can never pass as a real report.
<code>preview/scripted-step-feed.tsx</code>	69	Replays a recorded run for interface testing.
<code>ui/*.tsx</code>	419	Nine small building blocks — button, input, table, badge, alert, label, separator, skeleton, textarea. Standard components, restyled to the project's design system rather than used as shipped.

Part 3 — The database

One file, `db/schema.sql`, defines eight tables on Supabase (PostgreSQL).

TABLE	WHAT IT STORES
<code>companies</code>	Resolved company identities — CIK, ticker, filer type, industry code, currency.
<code>filings</code>	The filing manifest: which documents each report may draw on.
<code>reports</code>	One row per report — its status, ticker, depth and timings.
<code>facts</code>	Every sourced figure. All five provenance columns are declared NOT NULL, so the database itself refuses a fact without a source. This is the same rule as the code gate, enforced a second time one layer down.
<code>market_cache</code>	Cached market data, so a repeated report does not re-fetch prices.
<code>run_logs</code>	Every pipeline step with its state, duration and output count. Feeds the live progress screen and the metrics endpoint.
<code>artifacts</code>	Links to the generated PDF and Excel files.
<code>chat_messages</code>	The conversation history for a report's assistant.

Part 4 — Supporting files

FILE	WHAT IT DOES
<code>backend/tests/</code>	457 automated tests. Concentrated on the two modules where correctness is non-negotiable: the arithmetic module and the verification gate.
<code>backend/pyproject.toml</code>	Python project settings, type-checking and linting rules.
<code>backend/requirements.txt</code>	Python dependencies, pinned to exact versions.
<code>backend/Dockerfile</code>	How the backend is packaged for deployment.
<code>frontend/package.json</code>	Frontend dependencies and the build, lint and typecheck commands.
<code>.env.example</code>	An annotated list of every environment variable, with no real values.
<code>CLAUDE.md</code>	The project's binding rules — architectural constraints, not preferences.